

CSC 330 Programming Languages

Lecture #5

Language Design

There are over 700 different programming languages. At least according to a website, although I think that number is a bit too high. Regardless there are a whole lot of them. Fortunately, there are a lot less languages that are popular enough to be in constant use today, so no one is expecting you to know all 700 languages. However, you should be aware of the strengths and weaknesses behind the most common ones. Even if you don't know a particular language, when faced with a task if you at least know a language is good for that task, you can spend time learning it.

When designing a programming language, tradeoffs must be made. It is not possible to have a single language do everything and do it well. When a language is easy to learn, it might not be that powerful or efficient. And to get a language to be very efficient at a particular task means creating keywords and constructs suited for that task. This makes the language poorly suited for other things. Designing a language is very difficult and tradeoffs are inevitable.

Designing a Language

There may be hundreds of languages, but if designing a new language was easy, there would be thousands. Truth is it is very difficult to create a language and have it become popular. Many languages have been created and then mostly abandoned or have a very small following. Here are some design principles and things to keep in mind when examining programming languages.

Enhancement Proposal Systems

Several languages have an enhancement proposal system. For example, Python has PEP¹, which stands for Python Enhancement Proposal. These are proposals for extending the language or otherwise enhancing it. There are several of them including:

- PEP 0 – An index for all other PEP proposals.
- PEP 8 – A style guide governing how much white space one should use for indentation, when and where to put spaces, total number of characters per line and more.
- PEP 20 – the Zen of Python.
- PEP 257 – Conventions to use for documenting methods using "docstrings".
- PEP 596 – Release schedule for Python 3.9

And a lot more. There are hundreds of these spread over several categories. Users can read and comment on these proposals before they are solidified in the Python language. This is a great way to receive feedback on potential improvements to the language. Java has this as well, calling its proposals JEPs².

Borrowing from other languages

When Java was created, it was similar to C and C++. This was not by accident. C and C++ were very popular languages and to make a popular language yourself, you should start by copying other popular languages. JavaScript was named that since Java was a popular language at the time. C# borrowed several concepts from

¹ <https://peps.python.org>

² <https://openjdk.org/jeps/0>

C++ and Java for version 1.0. In general, developers don't want to start all over learning a brand new language. They would much rather carry over their knowledge into a new language. Sure, that language can have plenty of new and unique features, but the languages that end up being the most popular are the ones where the core aspects are already familiar to devs. This is why instead of having languages that are all completely different, we instead have groups of languages with similar if statement syntax, and while loop syntax, etc.

For the most part, the standard data types are very similar among languages as well. Object oriented languages all use the keyword "class". There are plenty of similarities with programming languages that help a developer make a smoother transition going from one language to another. Granted there are many differences as well and as languages evolve, sometimes the differences become more apparent, but a lot of the basics remain mostly the same. For example, when one language gets lambda functions, several other follow suit with mostly similar syntax. It's common for a new feature to come out in a language and that same feature makes its way to other languages.

Principle of Least Astonishment

This principle basically states that when designing a language, you should do the thing that least astonishes a developer. For example, if you add two ints, it makes sense to produce an int as a result. If a language decided that every time two ints were added a float would be produced, that would confuse (or "astonish") many developers and so that wouldn't be a good design decision.

This principle applies to general software design as well, especially in UX/UI. You should do the thing that least astonishes the user.

Coding Style

Developers normally have their own preferences when it comes to coding style. Part of that style pertains to naming convention. Each convention has its own name which is important to know.

camelCase	initial lowercase letter, subsequent words start with a capital letter
PascalCase	initial uppercase letter, subsequent words start with a capital letter
flatcase	all lowercase letters
UPPERCASE	all uppercase letters
snake_case	all lowercase letters, words separated with underscores
SCREAMING_SNAKE_CASE	all uppercase letters, words separated with underscores
kebab-case	all lowercase letters, words separated with dashes
SCREAMING-KEBAB-CASE	all uppercase letters, words separated with dashes

Most of the time, developers of a particular language will choose a certain style for variable, functions, classes, etc. This is either seen in the official style guides for a language or adhered to by common convention. It's important to note the common styles of each language:

	Variables	Functions	Classes	Constants
Python	snake_case	snake_case	PascalCase	SCREAMING_SNAKE_CASE
C++	snake_case camelCase	snake_case camelCase	snake_case PascalCase	SCREAMING_SNAKE_CASE
Java	camelCase	camelCase	PascalCase	SCREAMING_SNAKE_CASE
C#	camelCase	PascalCase	PascalCase	SCREAMING_SNAKE_CASE

Tradeoffs

There are several aspects to consider when designing a language. Normally, focusing on one means sacrificing another. Here are some aspects a language can focus on:

Ease of Learning

No language aims to be difficult to learn, however some specifically focus on being easy. Python, for example, was built to be an easy language to learn. Normally making aspects of a language easy to use means anticipating the most common use case for a construct and optimizing it for that. For example, Python's for loop is meant to run over a collection. Python makes it very easy to use a for loop for this. Consider Python's for loop vs. Java's:

```
for number in numbers:    # Python
for (int i = 0; i < numbers.length; i++) {    // Java
```

Python's for loop has a lot less boilerplate code and is therefore easier to use. However, Java's for loop is much more powerful, being able to be used for pretty much any condition with or without a list.

As another example, consider an efficient function definition in Java:

```
int[] add(int[] array, int value);    // Java
```

Java will automatically send the array argument in the most efficient way and will return an array efficiently. To do this, it makes a lot of assumptions that may or may not be true (or desirable). C++, on the other hand, is a little different. To achieve efficiency in C++, our function definition would look like this:

```
const int* add(int* const array, const int& value) const;    // C++
```

C++ gives you the ability to specify parameters as a value, a pointer, or a reference. Each of these has pros and cons and an experienced developer must know the difference between them. C++ also demands you use the "const" keyword everywhere it applies for maximum efficiency. There are several places the const keyword can be put in a function header. Again, an experienced C++ developer should know all of them. This makes even simple function headers seem a lot more complex. C++ is NOT an easy language to learn, but you get far more flexibility and control. Only through sacrificing some of this can you make a language easier to use.

Runtime Speed

C and C++ are the gold standard on runtime efficiency. But why is this? Consider the following code in C++:

```
int[] nums = { 1, 2, 3 };
cout << nums[4];
```

C/C++ do not do a lot of checks at runtime. In fact, they do very few! The code above will obviously result in an index out of bounds error. But C will not check this. It will just crash with a segmentation fault, giving the developer no helpful information at all. The reason why languages like Python and Java can give you a helpful error message for this runtime mistake, is because every array keeps track of its own length. This length is then checked every single time you access an element. If you go out of bounds, an exception object is created that has a lot of information in it. The problem is that this takes a lot of time! Imagine if every time you access

an element of an array, an if statement is ran first to check the length of that array. It would turn code like this:

```
for(int i = 0; i < SIZE; i++) {  
    cout << nums[i];  
}
```

into code like this:

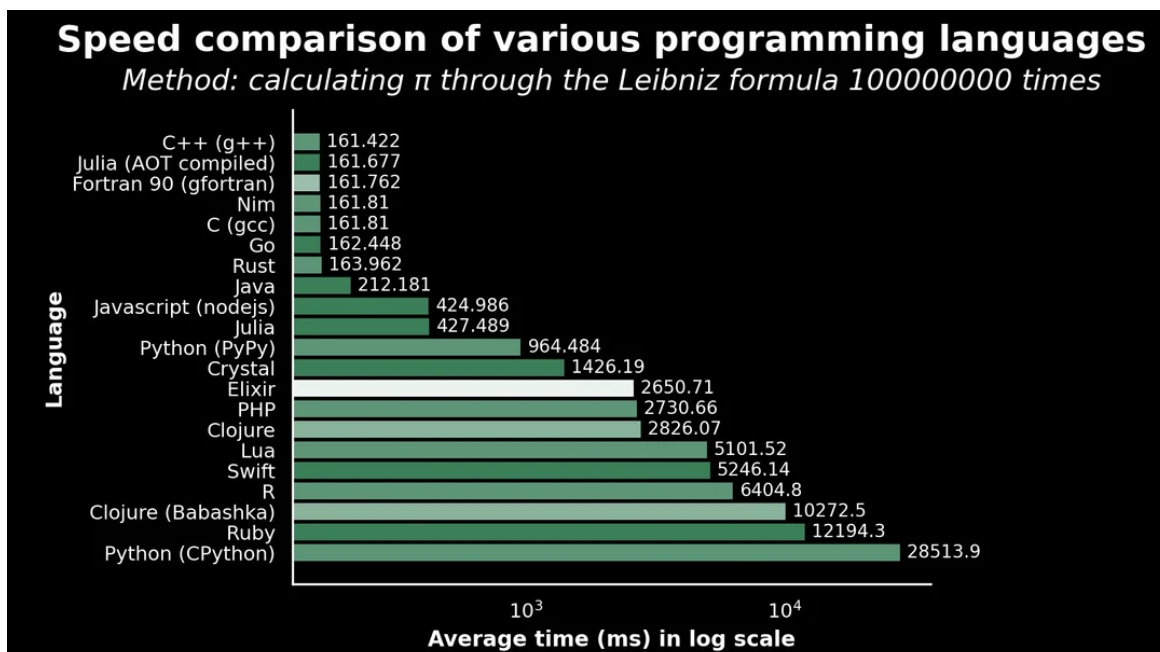
```
for(int i = 0; i < SIZE; i++) {  
    if (i < nums.length) {  
        cout << nums[i];  
    }  
    else {  
        // gather information and throw exception  
    }  
}
```

This is exactly what Java and Python are doing. We are trading array access safety for efficiency.

C and C++ do not have checks like this. In fact, you would be surprised just how many checks languages like Java and Python have at runtime that C/C++ doesn't have. This leads to large time differences. So, what's more important to you, safety and good error messages or speed?

Another aspect of programming languages that affect speed is its choice of translation. Interpreted languages will always be slower than hybrid languages, and hybrid languages will always be slower than fully compiled languages.

One thing to note is that you don't automatically get faster code by switching programming languages. This is something that I thought for many years and plenty of other people do to. Just by taking code in a slower language and rewriting it in C, doesn't make it faster by default. You need to be a proficient developer in C and really understand how to take advantage of all the ways you can make your code faster. You have the potential to write really fast code, but you have to have the knowledge and skill.



Standard Library

All (or almost all) programming languages give you the ability to create your own data types, whether it is a new class, struct, enum, etc. However, there are many data types that are used very commonly by developers. For this reason, languages come with a standard library. This could be things like `java.util.ArrayList` and `java.util.HashMap` for Java. Or `import random` and `import time` for Python. Or `#include <stdio.h>` for C.

Some standard libraries try to include everything the language developers can think of with all kinds of helper functions and classes. Others give you the basics and rely on you to create any helper methods you may need.

Language Extensibility

Some languages give you the ability to extend the language more than just adding data types. For example, C# gives you the ability to add methods to pre-existing classes. Even if it is a class that comes with the language. These are called "extension methods". For example, let's say you wanted to add something to the base string class that will count the number of occurrences of a specific character. You could write the following:

```
// separate class for all of your extension methods
public static class MyExtensions {
    // will be added to the string class
    // using the keyword "this" for the first parameter makes it
    // an extension method
    public static int CountOf(this string str, char target) {
        int count = 0;
        foreach (char c in str) {
            if (c == target) {
                count++;
            }
        }
        return count;
    }
}

public class Program {
    public static void Main(string[] args) {
        // create a normal string
        string test = "Extension Method";

        // call the method just like any other string method
        System.Console.WriteLine(test.CountOf('e')); // prints 2
        System.Console.WriteLine(test.CountOf('s')); // prints 1
    }
}
```

This gives you the ability to add any method to any pre-existing class and will be in effect for all files in the project.

The downside to this is that it can be abused. We should only put methods into a class if they have a somewhat common occurrence in our project. Remember that other developers aren't likely to know that a new method was added to a class, and so they might recreate the functionality in another area of the project.

C++ has macros that allow for extending the language. These are commands to the compiler that allow it to find and replace in a really powerful way. For example, we could add a new type of loop:

```
#include <iostream>
using namespace std;

// our macro. defines an "until" loop
#define until(x) while(!(x))

int main() {
    int count = 0;
    // compiler replaces this with:
    // while(!(count == 10))
    until(count == 10) {
        cout << count << endl;
        count++;
    }
}
```

Using macros I was able to create an "until" loop, which iterates until its condition is true.

Macros can also be abused. For example, if we added the following macro to the above code, bad things happen:

```
#include <iostream>

// our macro. defines an "until" loop
#define until(x) while(!(x))

// do NOT do this
#define int char

using namespace std;

int main() {
    int count = 0;
    // compiler replaces this with:
    // while(!(count == 10))
    until(count == 10) {
        cout << count << endl;
        count++;
    }
}
```

When attempting to compile the code, we get this error message:

```
$ g++ macros.cpp
macros.cpp:9:10: error: '::main' must return 'int'
    int main() {
        ^
```

This is because our macro changed all "int" into "char". A powerful feature but a deadly one as well!

Python has its own extensibility mechanism with decorators. You have used decorators before with accessors and mutators. To remind you, here is some code:

```
class Something:
    def __init__(self, initValue):
        self.someField = initValue

    @property
    def someField(self):
        return self._someField

    @someField.setter
    def someField(self, value):
        self._someField = value

s = Something('x')
s.someField = 'y'
print(s.someField)
```

If you are curious how this actually works behind the scenes, look up the Property class in Python (here's a good start: <https://realpython.com/python-property/>).

A decorator is a function that takes a function as a parameter and returns a new function (which is normally the given function but altered in some way). It gives you the ability to change any function's behavior. For example, I could change the name of a function:

```
# decorator that changes the name of
# any function to "bar"
def changeName(func):
    func.__name__ = 'bar'
    return func

def fool():
    return 3

@changeName
def foo2():
    return 3

print(fool.__name__) # outputs "fool"
print(foo2.__name__) # outputs "bar"
```

The @ symbol is really just syntactic sugar for the following:

```
# decorator that changes the name of
# any function to "bar"
def changeName(func):
    func.__name__ = 'bar'
    return func

def fool():
    return 3
```

```
def foo2():
    return 3

# this is what the @ symbol does for you
foo2 = changeName(foo2)

print(foo1.__name__) # outputs "foo1"
print(foo2.__name__) # outputs "bar"
```

You can use decorators to inject custom code before and after function execution:

```
def executionAlert(func):
    def newFunction():
        print("before execution")
        func()
        print("after execution")
    return newFunction
```

```
@executionAlert
def foo():
    print("function is running")
```

```
foo()
```

```
# Output:
#   before execution
#   function is running
#   after execution
```

You can even time a function to see how long it takes to run:

```
from time import time
```

```
def timeMyFunction(func):
    def newFunction():
        start = time()
        func()
        stop = time()
        print(f"Your function took {stop - start} seconds to execute")
    return newFunction
```

```
@timeMyFunction
def foo():
    for _ in range(100_000_000):
        pass
```

```
foo()
```

```
# Output:
#   Your function took 0.9831762313842773 seconds to execute
```

Special v. General purpose

Most languages you know of and have learned are general purpose languages. This means that you can use them for several different types of projects. For example, Python can be used for desktop applications, machine learning, scientific applications, 2D games (with pygame) and even web development (with Flask). Making a language general purpose means that most of its constructs and features must focus on lower level tasks, such as moving memory around, doing computations, or describing data types. If Python had a keyword or built-in function for displaying a texture to the screen, this wouldn't be very useful when developing a console/terminal app. This means that a lot of general purpose languages will push the responsibility for such special features off to library developers. The makers of the pygame library do have functions for displaying textures, since that library contains the needed functionality for making 2D games.

On the other hand, special purpose languages tend to go all in with constructs, features and keywords aimed at their specific task. For example, the language Matlab has a built in function to get the eigen values of a matrix. Matlab focuses on matrix operations and has several features that you won't find in a general purpose language. This makes the language very well suited for its specific tasks, but as a tradeoff you are highly unlikely to write a game or a website in it. Another example is the language GLSL, which is OpenGL's shading language. This language is meant to run on the GPU and contains several built in features for displaying colors on the screen. If you were to take a general purpose language and try to manually draw an image with it, you would need to create a 2D array of Color objects, spawn several threads, have each one working on different parts of the array, perform several matrix operations as you grab attributes from the vertices of a 3D object, and much more. In GLSL, most of this is either done for you or is trivial to do. Special purpose languages can only be used for their intended types of applications, but they are very powerful for that purpose.